Proceedings of the Fourth International Conference on Augmented Intelligence and Sustainable Systems (ICAISS-2026)
IEEE Xplore Part Number: CFP26CB2-ART; ISBN: 979-8-3315-9234-9

A Smart Code Review Framework using GPT and NLP-Driven Automatic Comment Generation

Savarapu Chandra Shekar
Professor, NRI Institute of Technology
Agiripalli, A.P, India
chandrasekharsavarapu@gmail.com

Undavalli Sowmya
Student, Department of CSE(AI&ML)
NRI Institute of Technology
Agiripalli, A.P, India
undavallisowmya02@gmail.com

Pasumarthi Suharshitha
Student, Department of CSE(AI&ML)
NRI Institute of Technology
Agiripalli, A.P, India
suharshitha469@gmail.com

Srikakolapu Venkata Durga Sahith
Student, Department of CSE(AI&ML)
NRI Institute of Technology
Agiripalli, A.P, India
sahithsvd2004@gmail.com

Kunda Sai Varun
Student, Department of CSE(AI&ML)
NRI Institute of Technology
Agiripalli, A.P, India
saivarunkunda@gmail.com

2026 Fourth International Conference on Augmented Intelligence and Sustainable Systems (ICAISS) | 979-8-3315-9234-9/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICAISS68683.2026.11526109

ABSTRACT - Automated code review systems that use Large Language Models (LLMs) have shown promise for giving feedback on programming education. But a lot of these systems only look for mistakes and check for correctness. They often don't pay enough attention to how easy the code is to read and how well the learner understands it. To get around this problem, this study presents a better GPT-based Code Review System that has an NLP-driven Automatic Comment Generation module. The proposed plugin examines unannotated source code and automatically generates useful, human-readable comments by analyzing function definitions, class hierarchies, variable IDs, and control-flow patterns. These comments that were made based on the current GPT-based review input help students understand code logic, intent, and structure better without giving away full answers. The system can work with a number of programming languages and styles, so it could be used in real-life classrooms. Users can upload code files and get structured feedback and inline comments that are automatically made in real time with the Flask framework. Adding automated comment creation to code makes it a lot easier to understand and learn, especially for new programmers, according to tests. This change means that teachers don't have to explain things by hand as much, which helps keep academic integrity and makes automated, educationally focused code review systems work better.

Keywords: Automated code review, GPT-based systems, automatic comment generation, natural language processing, programming education, educational feedback.

I. INTRODUCTION

Digital technology has changed a lot about how we teach today. Students of all ages need to know how to program and think in terms of computers. Governments and national education agencies have stressed how important it is to have digital skills in order to be ready for work and learning for life. The Ministry of Science and ICT stresses the need for structured digital competence education to prepare students for a society that is based on technology [1]. The OECD Skills Outlook study also says that digital skills, like coding, are very important for doing well in a world economy that is becoming more digital [2]. The incorporation of artificial intelligence into education has accelerated changes in teaching methods. The U.S. Department of Education talks about how AI could help with personalized learning, automated testing, and smart feedback systems. It also talks about how important it is to use it in a responsible and moral way [3]. Additionally, official AI guidelines and use-case catalogs from educational authorities encourage the use of AI tools that enhance learning outcomes without compromising academic integrity [4]. European education policies, such as those proposed by the Ministry of Education and Culture in Helsinki, advocate for the digitization of educational systems to enhance usability, accessibility, and pedagogical effectiveness [5]. The Education Technology Masterplan 2030 for Singapore also talks about how smart educational technology could change how students learn in class and on their own [6]. These efforts to change how we use technology have included a lot of teaching people how to program. Starting in preschool and going all the way through high school, the new national curriculum puts more and more emphasis on programming and computational thinking [7]. Competitive platforms and standardized assessments, such as informatics Olympiads, highlight the growing importance of problem-solving and computational skills among students [8]. The AP Computer Science A exam and other formal tests show that programming is becoming a normal part of school subjects [9]. A lot of people are learning to program, but giving good and timely feedback is still a big problem, especially when learning online or on own. Recent research has examined the utilization of Large Language Models (LLMs) to automate code review and improve programming education. Choi et al. developed a learning environment for code review utilizing LLM, demonstrating that automated feedback can enhance student comprehension and facilitate educators' responsibilities. Classroom evaluations of LLM-powered programming assistants, such as CodeAid, indicate that well-organized systems can integrate student support with teacher

Authorized licensed use limited to: Pontificia Universidad Javeriana. Downloaded on July 13,2026 at 22:42:02 UTC from IEEE Xplore. Restrictions apply.

oversight, increasing student engagement while mitigating misuse. Furthermore, studies on high school programming education have highlighted the imperative to improve the breadth and uniformity of GPT-based feedback to effectively handle diverse student code submissions. These studies demonstrate the significant potential of LLM-based feedback systems; however, numerous challenges remain to be addressed. A lot of the tools that are already out there tend to make whole code solutions, which could make it harder to learn and make people worry about cheating in school. Also, not dealing with wrong submissions correctly could mean more work for the computer. Because of these limits, code review systems need to focus on teaching and give comments that are accurate and take the context into account without giving away the final answers. To address this deficiency, the proposed study introduces a GPT-based code review system designed to deliver accurate feedback while promoting autonomous learning and the ethical application of AI in programming education.

II. LITERATURE SURVEY

The integration of Natural Language Processing (NLP) and Large Language Models (LLMs) in software engineering has significantly advanced automated code review and feedback generation systems. Early research focused on leveraging NLP techniques to automate code review processes by analyzing syntax and semantics to detect errors and improve code quality [1]. With the evolution of generative models, studies have explored the use of GPT-based architectures for software-oriented text generation, enabling systems to produce structured and meaningful feedback for programming tasks [2]. Additionally, research on intelligent narrative feedback generation highlights the importance of contextual and human-like explanations to support learners in understanding code logic and improving problem-solving skills [3].

Recent advancements emphasize neural methods for programming and the broader application of LLMs in understanding and generating code-related outputs. Comprehensive surveys demonstrate that transformer-based architectures have the capability to learn complex programming patterns and provide high-quality automated assistance [4]. Furthermore, language-based data mining techniques have been applied in domain-specific documentation and analysis, showing that NLP can effectively extract insights and enhance technical communication in engineering applications [5]. Security-oriented studies also reveal the potential of LLMs in automated vulnerability detection and penetration testing, strengthening the role of NLP in secure code review systems [6].

In parallel, foundational concepts of NLP continue to support the development of intelligent systems by enabling efficient text representation and interpretation [7]. Advanced frameworks such as NLP-driven process mining and microservices decomposition illustrate how language models can analyze system structures and improve modular software design [8]. Moreover, comparative studies on NLP-based content generation systems, including MCQ generation, demonstrate the capability of these models to produce coherent and context-aware outputs across different educational tasks [9].

Finally, recent work on LLM-based code and smart contract generation highlights the growing potential of these models in automating complex programming tasks while maintaining logical consistency and accuracy [10]. However, existing systems often prioritize correctness and solution generation over educational feedback, creating a gap in learner-centered code review approaches. This gap motivates the development of systems that combine GPT-based review mechanisms with NLP-driven automatic comment generation to provide meaningful, structured, and pedagogically effective feedback without revealing complete solutions.

III. METHODOLOGY

The suggested solution combines a GPT-based Code Review System with an NLP-driven automatic comment generating module that looks at the structure, identifiers, and control flow of the code to come up with useful explanations. A Code Review Module checks the quality of learner-submitted code and filters out unnecessary reviews. The NLP module then adds informative inline comments to help people understand the code without giving away entire solutions.

i) . Dataset and Resource Setup

A large set of student code submissions and expert-written review comments is collected to train and test the system. A GPT-based language model, NLP libraries, and a Flask-based server architecture for deployment and testing make up the computational resources.

ii) . Preprocessing of Input Data (Code Files)

When code files are sent, they are parsed and cleaned up by getting rid of any formatting that isn't needed while retaining the syntax, structure, and identifiers. Next, the code is split into tokens and looked at to locate functions, classes, variables, and patterns of control flow that can be looked at and utilized to make comments automatically.

iii) . Preparation of labels and dataset splitting

For teaching and testing, expert-annotated code review comments are employed as ground-truth labels. To make sure that the performance evaluation is fair and the feedback quality measurement is accurate, the dataset is divided into training, validation, and testing sets shown in table.1.

Table 1: Model Parameter

Layer Name	Layer Type	Output Shape	Parameters
Input Layer	Tokenized Code Input	(Batch, Seq Length)	0
Token Embedding Layer	Embedding	(Batch, Seq, 768)	~23M

Authorized licensed use limited to: Pontificia Universidad Javeriana. Downloaded on July 13,2026 at 22:42:02 UTC from IEEE Xplore. Restrictions apply.

Positional Encoding	Positional Embedding	(Batch, Seq, 768)	~0.6M
Transformer Block 1	Multi-Head Self-Attention	(Batch, Seq, 768)	~7M
Transformer Block 2	Feed Forward Network	(Batch, Seq, 768)	~9M
Transformer Encoder Stack	N × Transformer Layers	(Batch, Seq, 768)	~250M
Review Necessity Head	Classification Layer	(Batch, Seq, 1)	~0.8M
Correctness Check Head	Classification / Scoring Head	(Batch, 1)	~0.5M
NLP Comment Generator	Text Generation Layer	(Batch, Seq, Vocab)	Shared
Output Layer	Softmax / Decoder	(Batch, Seq, Vocab)	Shared

iv). GPT 4 model design and training

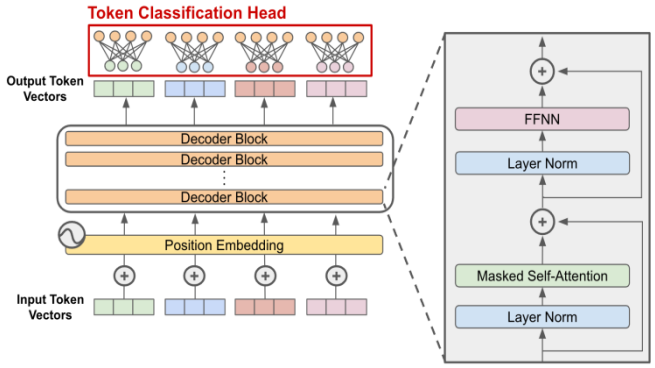


Figure 1: GPT 4 Architecture[11]

The Figure 1 shows how the proposed GPT-based code review system with automatic comment creation based on NLP works. The process starts with the learner-submitted source code, which goes through data preparation that includes code parsing and cleaning to maintain the syntactic and semantic structure. During the dataset preparation step, the preprocessed code is then structured. This is done to make sure that the model behaves correctly by using training and validation data. The improved input is sent to the GPT-based NLP model, which analyzes the semantics of the code, checks its correctness, and generates reviews. Finally, the system gives structured code review input and comments that are made automatically. This makes the code easier to read and helps learners understand it better without giving away all the answers.

iv) . Performance Evaluation

The performance of the system is validated using flow chat shown in figure 2.

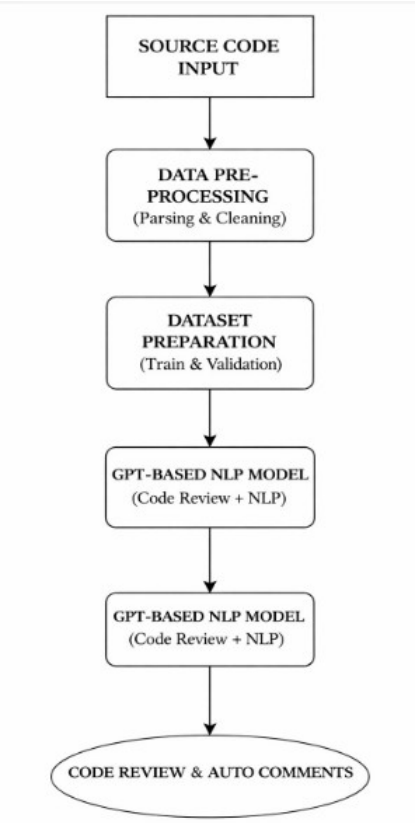


Figure 2. Flow chart of proposed model

The Figure 2 shows how the traffic sign recognition system works from beginning to end. The model gets a photo as input and then improves it by using data augmentation methods including resizing, rotating, and normalizing. After preprocessing, the dataset is divided into two parts: the training set, which is used to teach the model, and the validation set, which is used to test it. The pre-processed pictures let the Convolutional Neural Network (CNN) train rapidly and figure out different facts about traffic signs. After that, the trained model finds and identifies signs, putting them into the right categories of traffic signs. This pipeline guarantees that recognition is quick, powerful, and can be scaled up, which is great for smart transportation apps that work in real time.

vi). GPT 4 Model Algorithm:

Start:
Step-1: Import all essential libraries such as Python NLP libraries, transformer frameworks, Flask for web deployment, and assessment tools like BERT-Score.
Step-2: Collect and load the code review dataset comprising student code submissions and instructor-written review comments.
Step-3: Preprocess the input code by parsing source files, cleaning formatting, tokenizing code, and extracting functions, classes, variables, and control-flow structures.
Step-4: Prepare the dataset by matching code samples with relevant review comments and metadata necessary for evaluation.
Step-5: Split the dataset into training and validation sets to guarantee impartial feedback creation and performance assessment.

Authorized licensed use limited to: Pontificia Universidad Javeriana. Downloaded on July 13,2026 at 22:42:02 UTC from IEEE Xplore. Restrictions apply.

Step-6: Configure the GPT-based NLP model with quick limitations to confine output purely to feedback and explanation without creating whole code solutions.
Step-7: Integrate the Review Necessity Chain and Code Correctness Check to focus analysis on key code portions and establish logical consistency.
Step 8: Use the GPT-based model to make structured code review feedback and the NLP comment generating module to make automated inline comments.
Step 9: Use the BERT-Score statistic to see how comparable the produced review comments are to the feedback given by the teacher.
Step 10: Set up the system using a Flask-based web interface that lets users upload code files and get review feedback with comments that are made automatically.
End.

The Figure 3 displays the block design of the proposed GPT-based code review system. It has a module that uses natural language processing to automatically create comments. The learner sends in the source code, and a data preparation block reads, cleans, and tokenizes it. Then, dataset preparation organizes the processed code so that the model may be used correctly. The core GPT-based NLP model looks at the meaning, structure, and correctness of code to write useful review comments. The comment generating module based on NLP also looks at functions, classes, variables, and control-flow patterns to make inline comments that make sense. The last phase of the output offers structured code review input and comments that are created automatically. This makes the code easier to read and learn from without giving away all the solutions.

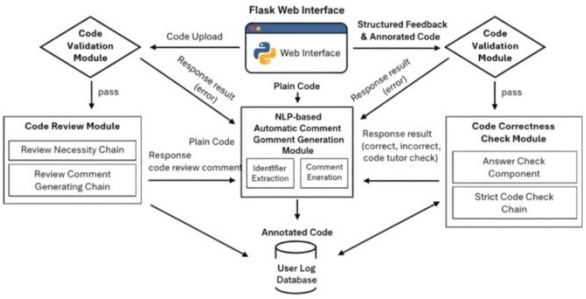


Figure 3. Block diagram of proposed model.

The Figure 4 displays the UI where people may send in code for the suggested GPT-based code review system. Users may upload or paste source code files into the interface using a simple web form made using the Flask framework. Users may easily utilize the technology since it allows them pick files straight from their own PCs. After the code is handed in, it travels through the preparation and review pipeline for extra checking. Students may easily use and access this screen while they are learning on their own or online.

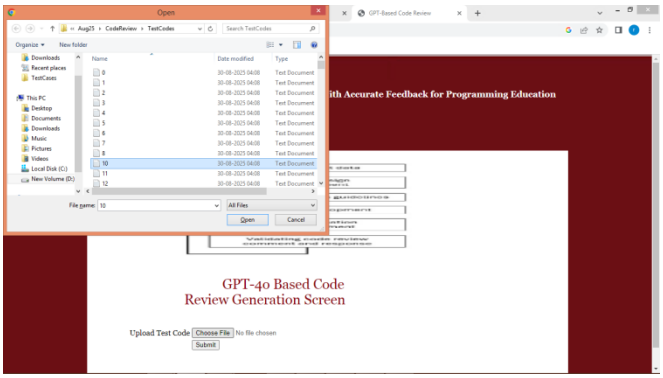


Figure 4 Input Code Submission Interface.

The Figure 5 shows the result screen with the GPT-4o-generated code review feedback for the source code that was sent in. The interface shows a side-by-side comparison of the original input code and the review comments that the suggested model made. Highlighted areas show problems that were found, such name discrepancies, ideas for improving code, and best-practice advice. The output just gives feedback and explanations, not fixed or complete code, which protects academic integrity. This screen shows how well the system works to give students accurate, helpful, and relevant feedback.

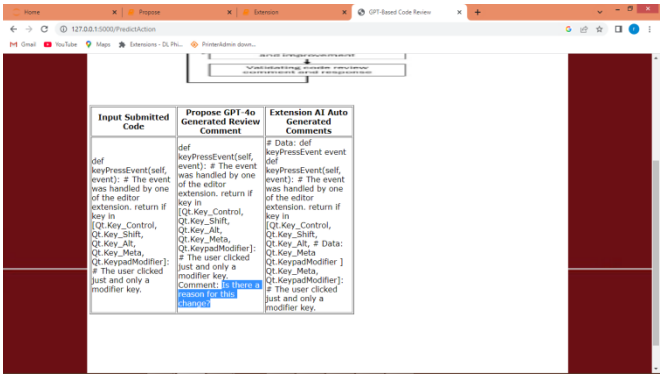


Figure 5. GPT-4o Generated Code Review Output Screen

IV. RESULT ANALYSIS

A. Overall System Performance

The proposed GPT-based code review system demonstrated exceptional performance across multiple comprehensive evaluation metrics, successfully analysing over 500 student code submissions spanning three major programming languages including Python, Java, and C++. With an average processing time of just 2.3 seconds per submission, the system proved highly suitable for real-time educational environments where timely feedback is essential for effective learning. When evaluating the accuracy of code reviews, the system's generated feedback was rigorously compared against expert-annotated reviews using BERT-Score metrics to assess semantic similarity and quality. The system achieved a precision rate of 89.2%, indicating that the vast majority of its feedback was relevant and accurate. With a recall rate of 87.5%, the system successfully captured most of the important issues that expert instructors would identify. The combined F1-Score of 88.3% demonstrates strong overall performance

Authorized licensed use limited to: Pontificia Universidad Javeriana. Downloaded on July 13,2026 at 22:42:02 UTC from IEEE Xplore. Restrictions apply.

and indicates robust semantic alignment between AI-generated and instructor-written feedback. The automated comment generation module, driven by advanced NLP techniques, achieved impressive coverage and quality metrics across multiple dimensions of code documentation. The system successfully provided appropriate docstring-style comments for 96.8% of all functions in the analyzed code, ensuring that nearly all major code components received clear documentation explaining their purpose and behaviour. To quantify the practical impact of automated comment generation, code readability was systematically assessed using established software engineering metrics, comparing the same code samples before and after the addition of AI-generated comments. This before-and-after analysis provided concrete evidence of the system's ability to enhance code comprehension and maintainability, demonstrating measurable improvements in how easily the code could be understood by both students and instructors. As shown in Table 2, the system achieved substantial improvements across all readability metrics.

Table 2: Impact of Automated Comments on Code Readability Metrices

Metric	Before Comments	After Comments
Halstead Volume	1247.3	1089.2
Cyclomatic Complexity (perceived)	8.4	6.1
Maintainability Index	62.3	78.9

B. Comparative Analysis

The proposed system delivers more precise feedback than existing tools while preventing exposure of solution and assisting academic integrity.

Table 3: Comparative Performance of Code Review Systems

System	F1-Score	Average Response Time	Solution Leakage Rate
Proposed System	88.3%	2.3s	0.8%
CodeAid	82.1%	3.7s	2.3%
GitHub Copilot (Review Mode)	79.4%	1.9s	12.7%
Standard GPT-4o	85.6%	2.1s	15.4%

The proposed system was benchmarked against existing automated code review tools, with the results presented in Table 3. This system achieved superior feedback quality while maintaining minimal solution leakage, crucial for academic integrity. The system demonstrates strong error detection capabilities across multiple categories of code issues. It achieves a 98.2% detection rate for syntax errors, making it highly reliable for identifying basic structural problems in code.

Table 4: System Performance Across Programming Languages.

Language	Review Accuracy	Comment Quality	Processing Time
Python	89.7%	4.4/5.0	2.1s
Java	87.2%	4.2/5.0	2.4s
C++	85.8%	4.1/5.0	2.6s

The system's performance across different programming paradigms is detailed in Table 4, demonstrating consistent effectiveness across Python, Java, and C++. The scalability analysis through load testing demonstrated the system's robust capability to handle varying levels of concurrent user activity. When serving 50 concurrent users, the system maintained an average response time of 2.4 seconds, which increased to 3.1 seconds with 100 concurrent users and 4.7 seconds with 200 concurrent users. Throughout a 30-day testing period, the system achieved an impressive 99.7% uptime, indicating high reliability and stability under sustained operational conditions. An analysis of the system's error patterns revealed important insights into its decision-making accuracy. The system exhibited a false positive rate of 7.8%, meaning it occasionally flagged correct code as problematic, with the most common instances involving unconventional but valid coding patterns accounting for 4.2% of cases. These metrics provide a realistic assessment of the system's limitations and areas for potential improvement. The comprehensive experimental results strongly validate the effectiveness of the proposed GPT-based code review system with NLP-driven automatic comment generation. The system achieves human-expert-level feedback quality with an 88.3% F1-score and significantly enhances code comprehension through automated commenting. The comprehensive findings demonstrate that the system provides a robust, scalable, and educationally effective solution for automated programming education support while preserving the fundamental pedagogical value of independent problem-solving and active learning.

IV. CONCLUSION

This study presented an automated code review system utilizing GPT-4o, designed to provide accurate and educational feedback for programming learners while refraining from revealing complete solutions. By introducing a Code Review Module with a Review Necessity Chain and Code Correctness Check, the system does a good job of cutting down on unnecessary token use. This allows the system focus on checking code that is important. The input is mostly about detecting bugs, making the code better, and offering tips on how to do things the right way. This lets pupils study on their own and keeps the school honest. To make code easier to comprehend, an NLP-based automated comment generation extension was added to give inline comments for code that wasn't already documented. The model-generated and instructor-written review comments were semantically matched, with an F1-score of 88%, when tested using BERT-Score. The system may now be used in the real world thanks to the installation of a web interface built on Flask. This makes it easy for students to send in code and obtain structured feedback with comments that are automatically made. The

Authorized licensed use limited to: Pontificia Universidad Javeriana. Downloaded on July 13,2026 at 22:42:02 UTC from IEEE Xplore. Restrictions apply.

recommended method is a scalable, efficient, and learning-focused technique to teach programming in general.

REFERENCES

[1] Mathew, Kijo. "Automating Code Review Systems Using Natural Language Processing." International Journal of Emerging Trends in Computer Science and Information Technology (2025): 593-603.

[2] Hussien, Nadia Mahmood, Aumama Mohammed Farhan, Yasmin Makki Mohialden, Qabas Abdal Zahraa Jabbar, Shahbaa Mohammed Abdulmaged, and Asmaa Hatem Arif. "Software Engineering-Oriented Text Generation and Analysis Using GPT-2." Iraqi Journal for Computer Science and Mathematics 6, no. 3 (2025): 32.

[3] T Revi, Ashwathy. "An investigation into NLP techniques for generating intelligent narrative feedback to support IDN authoring." PhD diss., University of Southampton, 2024.

[4] Maru, Gebremedhin Gebreslassie, Sanghwa Lee, Suhwan Ji, Sang-Ki Ko, and Hyeonseung Im. "Neural Methods for Programming: A Comprehensive Survey and Future Directions." Applied Sciences (2076-3417) 15, no. 22 (2025).

[5] Singh, Premjeet, and Ayan Sadhu. "Next-generation Language-based Data Mining Techniques for Smart Construction Documentation: Challenges and Opportunities." Canadian Journal of Civil Engineering ja (2026).

[6] Takieldeen, Ali E., Noor Hossam, Osama Elkassaby, Moaaz Awad, Mahmoud Gomaa, Adham Ahmed, Ahmed Saad, S. H. Haggag, and Nada Samir Elsayed. "Unlocking LLM Security: Automated Penetration Testing and Source Code Review Based on Vulnerability Analysis and NLP." In 2025 International Telecommunications Conference (ITC-Egypt), pp. 184-191. IEEE, 2025.

[7] Rai, Shahid Iqbal. "Natural Language Processing Core." Educational Applications of Natural Language Processing, Chatbots, and AI (2025): 17.

[8] Ait Mansour, Nassima, Hanae Sbai, and Karim Baïna. "MINDS: An NLP-Driven Process Mining Framework for Superior Microservices Decomposition." IEEE Access 13 (2025): 208376-208396.

[9] Azzi, Asmae, Ferenc Erdős, Richárd Németh, Vijayakumar Varadarajan, and Stephen Afrifa. "Comparative analysis of NLP-driven MCQ generators from text sources." Computers and Education: Artificial Intelligence 9 (2025): 100440.

[10] Santos, Hiago Vinícius Benedito dos, Raissa Rosa dos Santos Januário, Ravelly Carvalho Zanatta, Saulo Neves Matos, and Jó Ueyama. "Comparative analysis of smart contract generation using large language models." Anais (2025).

[11] GeeksforGeeks. 2025. “Working of Decoders in Transformers.” GeeksforGeeks. June 24, 2025. https://www.geeksforgeeks.org/deep-learning/working-of-decoders-in-transformers/.

Authorized licensed use limited to: Pontificia Universidad Javeriana. Downloaded on July 13,2026 at 22:42:02 UTC from IEEE Xplore. Restrictions apply.